

AdaptiveAvgPool3d#

[https://pytorch.org/docs/stable/generated/torch.nn.AdaptiveAvgPool3d.html#to](https://pytorch.org/docs/stable/generated/torch.nn.AdaptiveAvgPool3d.html#torch.nn.AdaptiveAvgPool3d)

Description:

Applies a 3D adaptive average pooling over an input signal composed of several input planes. The output is of size $D \times H \times W$, for any input size. The number of output features is equal to the number of input planes.

`output_size` (Union[int, None, tuple[Optional[int], Optional[int], Optional[int]]]) – the target output size of the form $D \times H \times W$. Can be a tuple (D, H, W) or a single number D for a cube $D \times D \times D$. D, H and W can be either a int, or None which means the size will be the same as that of the input. Input: $(N, C, D_{in}, H_{in}, W_{in})$ or $(N, C, D_{in}, H_{in}, W_{in})$. Output: (N, C, S_0, S_1, S_2) or (N, C, S_0, S_1, S_2) where $S = \text{output_size}$.

Function Signature

```
class torch.nn.AdaptiveAvgPool3d(output_size)[source]#
```

Parameters

Shape:

```
forward(input)[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> # target output size of 5x7x9
>>> m = nn.AdaptiveAvgPool3d((5, 7, 9))
>>> input = torch.randn(1, 64, 8, 9, 10)
>>> output = m(input)
>>> # target output size of 7x7x7 (cube)
>>> m = nn.AdaptiveAvgPool3d(7)
>>> input = torch.randn(1, 64, 10, 9, 8)
>>> output = m(input)
>>> # target output size of 7x9x8
>>> m = nn.AdaptiveAvgPool3d((7, None, None))
>>> input = torch.randn(1, 64, 10, 9, 8)
>>> output = m(input)
```

Detailed Documentation

Documentation

Applies a 3D adaptive average pooling over an input signal composed of several input planes.

The output is of size $D \times H \times W$, for any input size. The number of output features is equal to the number of input planes.

`output_size` (Union[int, None, tuple[Optional[int], Optional[int], Optional[int]]]) – the target output size of the form $D \times H \times W$. Can be a tuple (D, H, W) or a single number D for a cube $D \times D \times D$. D, H and W can be either a int, or None which means the size will be the same as that of the input.

Input: $(N, C, D_{in}, H_{in}, W_{in})$ $(N, C, D_{in}, H_{in}, W_{in})$ or $(C, D_{in}, H_{in}, W_{in})$
 $(C, D_{in}, H_{in}, W_{in})$ $(C, D_{in}, H_{in}, W_{in})$.

Output: (N, C, S_0, S_1, S_2) (N, C, S_0, S_1, S_2) or (C, S_0, S_1, S_2) (C, S_0, S_1, S_2) , where
 $S = \text{output_size}$

Runs the forward pass.